Git Basics for Beginners

What is Git?

Git is a popular version control system.

It was created by Linus Torvalds in 2005, and has been maintained by Junio Hamano since then.

It is used for:

- Tracking code changes
- Tracking who made changes
- Coding collaboration

🚀 Git Basic Workflow

0. Initial git settings

```
git config -global user.name "your name"
```

```
git config -global user.email "your email"
```

```
saihan: ~/../../dummy-project master
→ git config --local user.email "sa[redacted]@gmail.com"
saihan: ~/../../dummy-project master
→ git config --local user.name "Abdullah Saihan"
```

1. Start Git in your project

```
git init
```

```
saihan: ~/../../dummy-project
→ git init .
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/saihan/coderepo/ostad/dummy-project/.git/
```

👉 This tells Git: “Start tracking this folder”

---

2. Create your file and start tracking it

```
app.py  U ×
app.py
1   print("Hello World !")
```

```
git status
```

```
saihan: ~/../../dummy-project master
→ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        app.py

nothing added to commit but untracked files present (use "git add" to track)
```

```
git app.py
```

```
git status
```

```
saihan: ~/../../dummy-project master
→ git add app.py
saihan: ~/../../dummy-project master
→ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   app.py
```

👉 `git add` means: “Hey Git, include this file in the next save”

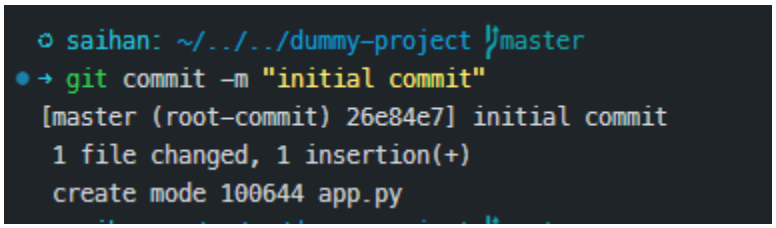
---

3. ✅ First Commit (VERY IMPORTANT)

```
git commit -m "add app.py"
```

Or

```
git commit -m "initial commit"
```

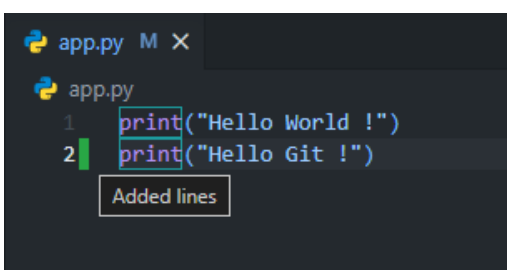


👉 This is your **first saved version** of the file
👉 Always commit right after you start tracking a file (good practice)

---

4. Make changes in your file

Edit [app.py]




```
git status
```

```
saihan: ~/../../dummy-project master
→ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py

no changes added to commit (use "git add" and/or "git commit -a")
```

👉 Shows: file is modified (you changed something, but not saved yet)

---

5. Prepare updated changes

```
git add app.py
```

```
git status
```

```
saihan: ~/../../dummy-project master
→ git add app.py
saihan: ~/../../dummy-project master
→ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   app.py
```

👉 Now your updated changes are ready to be saved

---

6. Save updated version

```
git commit -m "update app.py"
```

Or

```
git commit -m "add new line for greeting git"
```

```
saihan: ~/../../dummy-project master
→ git commit -m "add new line for greeting git"
[master 87db0c0] add new line for greeting git
 1 file changed, 2 insertions(+), 1 deletion(-)
```

```
saihan: ~/../../dummy-project master
→ git status
On branch master
nothing to commit, working tree clean
```

👉 Creates a **new version** with your latest changes

---

7. Check history

```
git log
```

```
git log --oneline
```

```
saihan: ~/../../dummy-project master
→ git log --oneline
```

```
87db0c0 (HEAD -> master) add new line for greeting git
26e84e7 initial commit
(END)
```

👉 Shows all versions

---

8. Go back to / check an old version

```
git checkout <commit_hash>
```

e.g.

```
git  checkout 26e84e7
```

👉 Lets you see how your project looked before

🌐 Push Code to GitHub

1. Create a repository on GitHub

👉 This is where your code will be stored online

---

2. Connect your project to GitHub

```
git remote add origin https://github.com/username/repo.git
```

👉 Links your local project with GitHub

---

3. Rename branch from master to main

```
git branch -M main
```

---

4. Push your code (first time)

```
git push -u origin main
```

👉 Uploads your project to GitHub

---

5. Next workflow (after changes)

git add .

git commit -m "your message"

git push

👉 Update your project online

🔥 Simple Daily Workflow

change code

git add .

git commit -m "what you changed"

git push

⚡ Key Tips

- Always **commit after adding new files first time**

- Commit frequently (small updates)
- Write clear messages
- Use `git status` to stay aware